

Universidad Técnica Federico Santa María

Ciencias de la Ingeniería II: Lenguajes de Programación

Certamen 1

Profesor: Alonso Díaz Candia

8 de mayo de 2026

Contexto

Un gimnasio de escalada en roca lleva un registro de sus rutas y los escaladores que las intentan. Cada ruta tiene un grado de dificultad y tres atributos que determinan qué tan exigente es: energía, técnica y fuerza. Los escaladores tienen sus propios niveles en esos mismos tres atributos, pero según su perfil los gastan y recuperan de forma distinta.

Se entrega el archivo `rutas.json` con los datos de las rutas disponibles en el gimnasio.

Puntaje

Parte	Descripción	Puntos
1	Clase Ruta	15
2	Clase Escalador	20
3	Clase EscaladorAficionado	25
4	Clase EscaladorElite	25
5	Programa principal	15
Total		100
6	Bonus	+10

1. Clase Ruta (15 puntos)

Implementa la clase `Ruta` con los siguientes atributos y métodos:

- Atributos públicos: `grado`, `tecnica`, `fuerza`.
- Atributo público `energia`: debe validarse en el setter. Si se intenta asignar un valor menor o igual a cero, se ignora el cambio.
- Atributo privado `_intentos`: parte en 0 y **no puede modificarse directamente desde fuera de la clase**. Solo se expone mediante una property de solo lectura `intentos`.
- Método `registrar_intento()`: incrementa `_intentos` en 1.
- `__str__`: retorna una cadena con el grado y la energía de la ruta. Ejemplo: `V3_21`.

2. Clase Escalador (20 puntos)

Implementa la clase base `Escalador`. Esta clase **no debe ser instanciada directamente** — existe para ser heredada por las clases de las partes 3 y 4.

- Atributos públicos: `nombre`, `tecnica`, `fuerza`.
- Atributo privado `_energia`: se asigna en el constructor y se expone mediante una property `energia`. El setter debe asegurarse de que la energía nunca quede bajo 0: si el valor asignado es negativo, se guarda 0.
- `contador_rutas = 0` y `grado_mas_dificil = None`: atributos públicos que se actualizan durante la simulación.
- `__str__`: retorna el nombre del escalador.

3. Clase EscaladorAficionado (25 puntos)

Hereda de `Escalador`. Representa a alguien que escala por recreación y tiene un límite de rutas que quiere intentar en el día.

- Atributo extra `limite_rutas`: número máximo de rutas que intentará. Si ya lo alcanzó, no intenta más aunque tenga energía.
- Método `puede_intentar(ruta)`: retorna `True` si **ambas** condiciones se cumplen:
 1. `energia >= ruta.energia`
 2. `contador_rutas < limite_rutas`
- Cuando escala una ruta, gasta un **10 % más** de energía que el valor base de la ruta (`ruta.energia * 1.1`).
- `__str__`: extiende el del padre agregando el límite de rutas. Ejemplo: `Carlos Lechuga (max 5 rutas)`.

4. Clase EscaladorElite (25 puntos)

Hereda de `Escalador`. Representa a un escalador de competición que tiene una especialidad y puede recuperar energía entre rutas.

- Atributo extra `especialidad`: puede ser "tecnica" o "fuerza".
- Atributo extra `recuperacion`: cantidad de energía que recupera después de cada ruta completada.
- Método `puede_intentar(ruta)`: retorna `True` si `energia >= ruta.energia`.
- Cuando escala una ruta:
 1. Si `especialidad == "tecnica"` y `tecnica >= ruta.tecnica + 10`, gasta un **20 % menos** de energía (`ruta.energia * 0.8`).
 2. Si `especialidad == "fuerza"` y `fuerza >= ruta.fuerza + 10`, gasta un **15 % menos** de energía (`ruta.energia * 0.85`).
 3. En cualquier otro caso, gasta la energía base de la ruta.

Después de gastar la energía, recupera `recuperacion` puntos.

- `__str__`: extiende el del padre con la especialidad. Ejemplo: `Sofia Dulce [tecnica]`.

5. Programa principal (15 puntos)

Implementa el flujo principal del programa:

1. Carga las rutas desde `rutas.json` y crea una lista de objetos `Ruta`.
2. Crea los siguientes escaladores:

Tipo	Nombre	Energía	Técnica	Fuerza	Extra
Aficionado	Pedro Roca	40	48	44	lím. 6 rutas
Aficionado	Luisa Vertical	55	62	51	lím. 8 rutas
Elite	Marco Cima	61	74	62	tecnica, rec. 3
Elite	Diana Cumbre	78	60	80	fuerza, rec. 5
Elite	Rafael Tope	84	59	81	fuerza, rec. 4

3. Para cada escalador, recorre las rutas en orden. Por cada ruta:

- Llama a `ruta.registrar_intento()`.

- Si `puede_intentar(ruta)` retorna `True`: descuenta la energía según el tipo de escalador, actualiza `contador_rutas` y `grado_mas_dificil`.
 - Si retorna `False`: deja de recorrer las rutas de ese escalador.
4. Al terminar, imprime para cada escalador cuántas rutas completó y cuál fue el grado más difícil que logró.

6. Bonus (+10 puntos)

Usando `filter` y `lambda`, obtén la lista de rutas cuya energía sea mayor a 40. Luego, usando `map`, extrae solo los grados de esas rutas e imprímelos.